



# ΕΠΕΞΕΡΓΑΣΙΑ ΦΩΝΗΣ ΚΑΙ ΦΥΣΙΚΗΣ ΓΛΩΣΣΑΣ

1Η ΕΡΓΑΣΤΗΡΙΑΚΗ ΑΣΚΗΣΗ (KALDI)

ΟΝΟΜΑ: ΗΛΙΑΣ-ΣΤΥΛΙΑΝΟΣ ΗΛΙΑΔΗΣ ΑΜ: 03122140

ΟΝΟΜΑ: ΣΤΑΥΡΟΣ ΜΑΝΙΑΤΟΓΙΑΝΝΗΣ ΑΜ: 03122427

## 1. ΘΕΩΡΗΤΙΚΗ ΕΙΣΑΓΩΓΗ

### 1. Mel-frequency Cepstral Coefficients (MFCCs)

Οι Mel-frequency Cepstral Coefficients είναι συντελεστές που εξάγονται από το mel-cepstrum και χρησιμοποιούνται σε συστήματα ανάλυσης φωνής. Ειδικότερα, το cepstrum είναι μία μη γραμμική μέθοδος ανάλυσης φωνής που επιλύει το πρόβλημα της αποσυνέλιξης. Το cepstrum βασίζεται στη σχέση

$$\hat{X}(z) = \ln[X(z)]$$

όπου  $X(z)$  ο μετασχηματισμός Z του σήματος  $x[n]$ .

Το cepstrum διαχρίνεται σε δύο κατηγορίες, το μιγαδικό-πλήρες και το απλό.

- Μιγαδικό cepstrum:  $\hat{x}[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} \hat{X}(e^{j\omega}) e^{j\omega n} d\omega$
- Cepstrum (Bogert-Healy-Tukey):  $\hat{x}[n] = \frac{1}{2\pi} \int_{-\pi}^{\pi} \log(|X(e^{j\omega})|) e^{j\omega n} d\omega$

Το πεδίο του cepstrum είναι το Quefrency και το  $\hat{x}[n]$  είναι μία φθίνουσα ακολουθία.

Το mel-cepstrum είναι μία ειδική μορφή cepstrum που βασίζεται στην κλίμακα mel και στον short-time Fourier transform (STFT). Η κλίμακα mel ορίζεται ως

$$\text{mel}(f) = 1127 \ln \left( 1 + \frac{f}{700} \right)$$

και μοντελοποιεί τις συχνότητες σύμφωνα με το πως οι άνθρωποι αντιλαμβάνονται τις διαφορές μεταξύ των συχνοτήτων. Προκειμένου να μετατρέψουμε ένα σήμα στην κλίμακα mel χρησιμοποιούμε την προσέγγιση των triangular filter banks, όπου κάθε φίλτρο είναι τριγωνικό με μέγιστη απόκριση τη μονάδα στην κεντρική του συχνότητα. Οι κεντρικές συχνότητες των φίλτρων είναι ομοιόμορφα κατανομημένες στην κλίμακα mel.

Για την ανάλυση του συχνοτικού περιεχομένου των σημάτων φωνής χρησιμοποιείται το φασματογράφημα που υπολογίζεται με βάση το μέτρο του μετασχηματισμού STFT. Ο STFT ορίζεται ως

$$X(m, \omega_k) = \sum_{n=-\infty}^{\infty} x[n] w[n-m] e^{-j\omega_k n}$$

όπου  $w[n]$  παράθυρο. Το παράθυρο, είναι ένα σήμα μικρής χρονικής διάρκειας που χρησιμοποιείται για την απομόνωση των frames από το αρχικό σήμα φωνής. Μάλιστα, λόγω της μικρής διάρκειάς τους, μπορούμε να τα θεωρήσουμε ως stationary διαδικασίες, παρότι τα σήματα φωνής είναι non stationary. Ειδικότερα, το κατάλληλο μέγεθος και είδος του παραθύρου, καθώς και η επιθυμητή επικάλυψη των πλαισίων (frame stride) εξαρτώνται από το εκάστοτε σήμα φωνής και το συχνοτικό του περιεχόμενο.

Στην πράξη, επιλέγουμε συνήθως smoothing παράθυρα, καθώς επιθυμούμε τα frames να μην έχουν υψηλές τιμές στα άκρα τους. Ένα τέτοιο παράθυρο, είναι και το Hamming window, που ορίζεται ως

$$w[n] = \begin{cases} 0.54 - 0.46 \cos\left(\frac{2\pi n}{L}\right), & 0 \leq n \leq L-1 \\ 0, & \text{otherwise} \end{cases}$$

όπου  $L$  το μέγεθος του παραθύρου.

Με βάση την έως τώρα ανάλυσή μας, γίνεται αντιληπτή η χρησιμότητα του mel-cepstrum σε συστήματα ανάλυσης φωνής, καθώς όχι μόνο οι συχνότητες βρίσκονται στην κλίμακα mel, αλλά και αρκεί μικρός αριθμός τελεστών για να αποθηκεύσουμε το συχνοτικό περιεχόμενο του αρχικού σήματος ανά τον χρόνο. Το mel-cepstrum ορίζεται ως

$$C_{mel}[n, m] = \frac{1}{R} \sum_{l=0}^{R-1} \log(E_{mel}(n, l)) \cos\left(\frac{2\pi}{R}(l + 0.5)m\right)$$

όπου  $R$  το πλήθος των mel φίλτρων.

Η  $E_{mel}(n, l)$  είναι η ενέργεια του  $l$  mel-scale φίλτρου τη στιγμή  $n$  και υπολογίζεται από τη σχέση

$$E_{mel}(n, l) = \frac{1}{A_l} \sum_{k=L_l}^{U_l} |V_l(\omega_k)X(n, \omega_k)|^2$$

όπου  $U_l$  και  $L_l$  ο μέγιστος και ο μικρότερος δείκτης συχνότητας όπου το φίλτρο είναι μη μηδενικό και

$$A_l = \sum_{k=L_l}^{U_l} |V_l(\omega_k)|^2$$

Οι Mel-frequency Cepstral Coefficients αποτελούνται πρακτικά από cepstral coefficients ( $c_t[i]$ ), για καθέναν από τους οποίους υπολογίζεται ένας delta cepstral coefficient ( $d_t[i]$ ) και ένας double delta cepstral coefficient ( $d_t^2[i]$ ) για κάθε frame  $t$ . Οι delta coefficients υπολογίζονται ως εξής

$$d_t[i] = \frac{\sum_{k=1}^K k (c_{t+k}[i] - c_{t-k}[i])}{2 \sum_{k=1}^K k^2}$$

$$d_t^2[i] = \frac{\sum_{k=1}^K k (dc_{t+k}[i] - dc_{t-k}[i])}{2 \sum_{k=1}^K k^2}$$

όπου  $K$  είναι η παράμετρος delta window.

Συνήθως, χρησιμοποιούνται 13 cepstral coefficients, 13 delta cepstral coefficients και 13 double delta cepstral coefficients, δηλαδή το MFCC αποτελείται κατά κανόνα από ένα διάνυσμα 39 χαρακτηριστικών. Σημειώνουμε ότι οι όροι  $c_t[0]$ ,  $d_t[0]$  και  $d_t^2[0]$  αναφέρονται και ως energy coefficient, delta energy coefficient και double delta energy coefficient αντίστοιχα.

## 2. Γλωσσικά Μοντέλα (Language Models)

Ως γλωσσικό μοντέλο ορίζουμε ένα μοντέλο μηχανικής μάθησης, το οποίο εκτιμά την πιθανότητα εμφάνισης μελλοντικών λέξεων σε ένα κείμενο. Δηλαδή, πρόκειται για ένα μοντέλο που σύμφωνα με κάποιο κριτήριο υπολογίζει την πιθανότητα μία λέξη ή μία πρόταση να ακολουθεί του τωρινού κειμένου. Για τον σκοπό αυτό, απαιτείται συνήθως το σύνολο των λέξεων, το λεξικό, να είναι γνωστό.

Τα γλωσσικά μοντέλα είναι ένα βασικό εργαλείο με το οποίο δύναται να εκτιμηθεί η συνοχή και η συνέπεια ενός κειμένου, ώστε να εντοπιστούν και να διορθωθούν τυχόν λάθη. Σε ένα κείμενο, μπορεί να υπάρχουν διάφοροι τύποι σφαλμάτων διαφορετικής δυσκολίας εντοπισμού. Λάθη γραμματικής, ορθογραφικά και τυπογραφικά, εντοπίζονται με μία αναζήτηση στο λεξικό και μπορούν να διορθωθούν από σχετικά απλά

γλωσσικά μοντέλα. Ωστόσο, ασάφειες και αντιφάσεις είναι πιο δύσκολο να αντιμετωπιστούν και συχνά απαιτούν προηγμένα γλωσσικά μοντέλα για να ανιχνευθούν και να διορθωθούν.

Συνεπώς, τα γλωσσικά μοντέλα μπορούν να περιορίσουν τα σφάλματα του κειμένου εισόδου και βρίσκουν εφαρμογή σε συστήματα επεξεργασίας φυσικής γλώσσας.

Υπάρχουν διάφορες κατηγορίες γλωσσικών μοντέλων, ορισμένα πιο απλά ως προς την υλοποίησή τους, όπως τα N-gram language models και άλλα περισσότερο περίπλοκα, όπως τα Large language models. Στην παρούσα εργασία όμως θα ασχοληθούμε με τα N-gram language models, τα οποία ορίζονται ως εξής.

Έστω  $w_{i+1}, w_{i+2}, w_{i+3}, \dots, w_{i+N-1}$  οι τελευταίες  $N - 1$  λέξεις του κειμένου. Τότε, η πιθανότητα η επόμενη λέξη του κειμένου να είναι η λέξη  $x$ , είναι ίση με:

$$\Pr[w_{i+N} = x] = \Pr[x \mid w_{i+1}w_{i+2}\dots w_{i+N-1}]$$

Τα N-gram language models είναι χρήσιμα σε περιπτώσεις όπου τα Large language models είναι δύσκολα υπολογιστικά, ωστόσο δεν χειρίζονται αποτελεσματικά τις συσχετίσεις μεταξύ λέξεων που εμφανίζονται απόσταση μεταξύ τους και δεν μοντελοποιούν ικανοποιητικά μη γνωστές ακολουθίες λέξεων.

Στην άσκηση, θα εργαστούμε με unigram ( $N = 1$ ) και bigram ( $N = 2$ ) μοντέλα.

Στην περίπτωση των unigram μοντέλων, οι πιθανότητες μπορούν να υπολογιστούν εύκολα ως  $\Pr[w] = \frac{\text{count}(w)}{\text{total words}}$ , ενώ για τα bigram μοντέλα  $\Pr[w_i \mid w_{i-1}] = \frac{\Pr[w_{i-1}w_i]}{\Pr[w_{i-1}]} = \frac{\text{count}(w_{i-1}w_i)}{\text{count}(w_{i-1})}$ . Βέβαια, για την αποφυγή underflow λόγω συνεχόμενων πολλαπλασιασμών με μικρούς αριθμούς, οι πιθανότητες υπολογίζονται και αποθηκεύονται σε λογαριθμική μορφή.

Ωστόσο, τα N-gram language models παρουσιάζουν καλές επιδόσεις μόνο εάν το test set είναι πανομοιότυπο με το train set. Ακόμα βέβαια και σε αυτές τις περιπτώσεις, δύναται να υπάρξει στο test set κάποια ακολουθία λέξεων που δεν παρουσιάζεται στο train set, με αποτέλεσμα το μοντέλο μας να έχει αναθέσει λανθασμένα μηδενική πιθανότητα στη συγκεκριμένη ακολουθία. Αυτός είναι και ο βασικός λόγος για τον οποίο τα N-gram language models δεν παρουσιάζουν σθεναρότητα και δεν είναι κατάλληλα για γενίκευση.

Προκειμένου να αποφύγουμε τις μηδενικές πιθανότητες, αναθέτουμε με βάση κάποιο κριτήριο (π.χ. add-one, interpolation, backoff) μικρή πιθανότητα στις ακολουθίες που δεν παρουσιάζονται στο training set.

Για την αξιολόγηση των γλωσσικών μοντέλων θα πρέπει να εκπαιδεύσουμε τα μοντέλα σε ένα train κείμενο να αξιολογήσουμε την επίδοσή τους σε ένα test κείμενο σύμφωνα με κάποια μετρική. Μία τέτοια μετρική είναι η perplexity (PP).

$$PP(w) = 2^{H(w)}, \quad H(w) = -\frac{1}{M} \log_2 P(w)$$

όπου  $M$  το πλήθος των λέξεων. Μάλιστα, ισχύει ότι

$$PP(w) = 2^{H(w)} = P(w_1, w_2, \dots, w_M)^{-\frac{1}{M}}, \quad \text{καθώς } M \rightarrow \infty$$

Σύμφωνα με τη συγκεκριμένη μετρική, όταν οι προβλέψεις των λέξεων γίνονται πιο ακριβείς, δηλαδή όταν αυξάνονται οι δεσμευμένες πιθανότητες, η τιμή perplexity ελαττώνεται. Συνεπώς, ένα μοντέλο αξιολογείται ως καλύτερο από ένα άλλο αν εμφανίζει μικρότερη τιμή perplexity.

Στην περίπτωση του unigram μοντέλου  $H(w) = -\frac{1}{M} \sum_{n=1}^M \log_2 P(w_n)$ , ενώ στην περίπτωση του bigram μοντέλου  $H(w) = -\frac{1}{M} \sum_{n=1}^M \log_2 P(w_n \mid w_{n-1})$ .

Η perplexity είναι μία intrinsic evaluation, χρήσιμη για την εξαγωγή συμπερασμάτων για την απόδοση των γλωσσικών μοντέλων, ωστόσο δεν είναι ασφαλής για την εκτίμηση της επίδοσης τους σε πραγματικές εφαρμογές.

### 3. Ακουστικά Μοντέλα

Ακουστικά μοντέλα ονομάζονται τα υπολογιστικά μοντέλα που δέχονται ως είσοδο χαρακτηριστικά που έχουν εξαχθεί από ένα σήμα φωνής και το αντιστοιχίζουν σε μια σειρά φωνημάτων (ή συλλαβών). Τα βήματα που ακολουθούνται συνήθως για την χρήση ενός Ακουστικού Μοντέλου είναι τα εξής:

1. **Απόκτηση Δεδομένων:** Αρχικά πρέπει να είναι διαθέσιμα τα Ηχητικά Σήματα που θέλουμε να αναλύσουμε.

2. **Εξαγωγή Χαρακτηριστικών από Ηχητικό Σήμα:** Σε αυτό το βήμα χρησιμοποιούμε διάφορες μεθόδους για να μετατρέψουμε την χρήσιμη πληροφορία του ηχητικού σήματος σε μορφή που είναι εύκολο να επεξεργαστεί. Συνήθως γίνεται μέσω MFCC (αναλύεται λεπτομερώς στο μέρος 1 της Θεωρητικής Εισαγωγής) και διάφορες παραλλαγές του που λαμβάνουν υπόψιν πιο συγκεκριμένες πληροφορίες ή περιορισμούς, ανάλογα με την εφαρμογή. Επειδή η μέθοδος αυτή είναι επιρρεπής στον θόρυβο, συνήθως γίνεται και ένα φιλτράρισμα του αρχικού σήματος.
3. **Αντιστοίχιση Χαρακτηριστικών σε Φώνημα:** Το Ακουστικό Μοντέλο δέχεται ως είσοδο τα χαρακτηριστικά του προηγούμενου βήματος και αποφασίζει για την ύπαρξη και ταυτοποίηση φωνημάτων για κάθε χρονικό πλαίσιο (συνήθως η διάρκεια ενός χρονικού πλαισίου ορίζεται στο εύρος 10ms-30ms). Το μοντέλο συνήθως χρειάζεται να εκπαιδευτεί με δεδομένα εκπαίδευσης για να μπορέσει να κάνει τις απεικονίσεις από χαρακτηριστικά σε φωνήματα, αλλά υπάρχουν και μοντέλα που έχουν ρυθμιστεί χειροκίνητα και επομένως δεν χρειάζονται εκπαίδευση.
4. **Αξιοποίηση των εξόδων Ακουστικού Μοντέλου:** Συνήθως τα αποτελέσματα του Ακουστικού Μοντέλου συνδυάζονται με τα αποτελέσματα ενός γλωσσικού μοντέλου, μέσω ενός Αποκωδικοποιητή, ώστε να συμπεράνουμε την πιο πιθανή πρόταση που αντιστοιχεί στα φωνήματα που εξάγαμε.

Σε αυτή την εργασία θα δούμε 2 μεγάλα είδη ακουστικών μοντέλων, τα GMM-HMM και τα DNN-HMM.

### GMM-HMM

Τα μοντέλα GMM-HMM (Gaussian Mixture Model - Hidden Markov Model) είναι από τα πρώτα και τα πιο διαδεδομένα Ακουστικά Μοντέλα. Μέσω του Μαρκοβιανού μοντέλου περιγράφει την χρονική δομή και τις μεταβάσεις καταστάσεων του λόγου και μέσω του Αθροίσματος Γκαουσιανών περιγράφει την πιθανότητα κάθε φωνήματος δεδομένου της τρέχουσας κατάστασης. Έτσι καταφέρνουμε να μοντελοποιήσουμε και τις ακολουθιακές και τις στοχαστικές ιδιότητες του προφορικού λόγου. Για κάθε νέα είσοδο (συνήθως MFCC feature vector) υπολογίζουμε την πιο πιθανή κατάσταση σύμφωνα με το HMM και δεδομένου της κατάστασης μπορούμε να υπολογίσουμε την πιθανότητα για το ποιο φώνημα ειπώθηκε σε αυτό το χρονικό πλαίσιο. Για την εκπαίδευση χρειάζεται να μάθουμε τις πιθανότητες μεταβάσεων καταστάσεων για το Κρυφό Μαρκοβιανό Μοντέλο και τις παραμέτρους και τα βάρη των Γκαουσιανών που αντιστοιχούν σε κάθε κατάσταση. Στην αποκωδικοποίηση ο στόχος είναι να βρεθεί, μέσω του δυναμικού αλγόριθμου Viterbi, η πιο πιθανή ακολουθία καταστάσεων και μέσω αυτής, η πιο πιθανή πρόταση στην οποία αντιστοιχεί το ηχητικό σήμα. Στα θετικά είναι πως είναι πολύ πιο υπολογιστικά αποδοτικό σε σχέση με άλλες μεθόδους, ωστόσο είναι πολύ απλό και κάνει ισχυρές υποθέσεις για την μορφή των δεδομένων, όπως ότι το άθροισμα Γκαουσιανών μπορεί να περιγράψει την μορφή της πυκνότητας πιθανότητας. Επομένως συχνά (και σε αυτή την άσκηση) δεν χρησιμοποιείται για την τελική επαγωγή συμπερασμάτων, αλλά δημιουργούνται ως ένα ενδιάμεσο βήμα για να κάνουν "bootstrap" άλλα πιο σύνθετα, αλλά και πιο υπολογιστικά δαπανηρά, μοντέλα, π.χ. μέσω aligning.

### DNN-HMM

Τα μοντέλα DNN-HMM (Deep Neural Network - Hidden Markov Model) είναι τα μοντέλα που χρησιμοποιούνται ως επί το πλείστον σήμερα, ειδικά όταν δεν έχουμε μεγάλους περιορισμούς πόρων σε σχέση με τους στόχους μας. Το HMM πάλι κωδικοποιεί την χρονική δομή, ενώ το DNN μπορεί να πάρει ως είσοδο ένα MFCC feature vector (όπως στο GMM-HMM), αλλά θα μπορούσε να πάρει μέχρι και το ίδιο το ηχητικό σήμα, προσφέροντας μεγαλύτερη ευελιξία στην εκπαίδευση, καθώς θα μπορεί να προσαρμοστεί καλύτερα στις απαιτήσεις κάθε εφαρμογής, χωρίς να πρέπει εμείς να βρούμε έναν κατάλληλο μετασχηματισμό για το τα διανύσματα χαρακτηριστικών. Επιπλέον το DNN τώρα βγάζει κατευθείαν στην έξοδο τις a posteriori πιθανότητες του κάθε φωνήματος. Στην εκπαίδευση το DNN ελαχιστοποιεί συνήθως το cross-entropy σε alignments που έχει πάρει από ένα GMM-HMM, δεν είναι απαραίτητο, αλλά χωρίς αυτό το κόστος υπολογισμού αυξάνεται ραγδαία και συχνά φτάνει απαγορευτικά επίπεδα. Στο κομμάτι της αποκωδικοποίησης δεν υπάρχει σημαντική διαφορά με το GMM-HMM.

## 2. ΔΙΑΔΙΚΑΣΙΑ ΚΑΙ ΑΠΟΤΕΛΕΣΜΑΤΑ

### 3. Βήματα προπαρασκευής

Έγινε μέσω Python Script. Για uttids επιλέχθηκε το <speaker>\_<file\_index> (π.χ. f1\_001), τα uttids είναι ένας ξεχωριστος δείκτης για κάθε αρχείο ήχου. Δεν θεωρήσαμε περαιτέρω ανάλυση απαραίτητη

για αυτό το μέρος

## 4. Βήματα κυρίως μέρους

### 4.1 Προετοιμασία διαδικασίας αναγνώρισης φωνής για τη USC-TIMIT

Έγινε μέσω Python Script. Το mfcc.conf πάρθηκε από το [lab2](#) του github των εργαστηρίων. Ακολούθηθηκαν οι οδηγίες της εκφώνησης χωρίς να χρειαστεί να αλλάξουμε κάτι, επομένως δεν θεωρούμε απαραίτητη περαιτέρω ανάλυση.

### 4.2 Προετοιμασία γλωσσικού μοντέλου

Για το ερώτημα **4.2.1** προκειμένου να συμπληρώσουμε τα αρχεία `lexicon.txt` και `nonsilence_phones.txt`, εξάγαμε τα φωνήματα από το αρχείο λεξικού που μας δόθηκε (`usc_lexicon.txt`), μέσω της εντολής `python "line.strip().split()"` και αποθηκεύσαμε τα φωνήματα σε ένα `set`. Ειδική διαχείριση απαιτήθηκε για τον χειρισμό της συμβολοσειράς "`<oov>`" (out of vocabulary), η οποία αν και δεν είναι φώνημα, πρέπει να βρίσκεται στο αρχείο "`lexicon.txt`" προκειμένου να μπορέσει να εκτελεστεί η εντολή του ερωτήματος 4.2.4. Για αυτόν τον λόγο αποφασίσαμε το αρχείο "`lexicon.txt`" να περιέχει την γραμμή "`<oov> sil`" αντί της "`<oov> <oov>`", ώστε η συμβολοσειρά "`<oov>`" να αντιστοιχίζεται στο φώνημα σιωπής και να μην χρειάζεται να συμπεριληφθεί σε κανένα από τα αρχεία `nonsilence_phones.txt`, `silence_phones.txt` και `optional_silence.txt`. Στο υπόλοιπο κομμάτι του κώδικα ακολουθούμε τις οδηγίες του ερωτήματος και δεν παρουσιάζεται κάτι άξιο σχολιασμού.

Στο ερώτημα **4.2.2** δημιουργούμε μέσω bash script ένα unigram και ένα bigram μοντέλο, ακολουθώντας τις οδηγίες της εκφώνησης.

Σύμφωνα με την εντολή που μας δίνεται στο ερώτημα **4.2.3**, παράγουμε τα compressed .arpa αρχεία `lm_phone_ug.arpa.gz` και `lm_phone_bg.arpa.gz`.

Για το ερώτημα **4.2.4** δημιουργούμε το FST λεξικό της γλώσσας χρησιμοποιώντας την εντολή του Kaldi `prepare_lang.sh` που βρίσκεται στον φάκελο `utils`. Η συγκεκριμένη εντολή δέχεται τα εξής τέσσερα ορίσματα:

1. directory του λεξικού
2. την συμβολοσειρά "`<oov>`" που εκφράζει ότι μία λέξη είναι άγνωστη
3. temporary lang directory
4. output lang directory

Για το ερώτημα **4.2.5** ταξινομούμε τα ζητούμενα αρχεία κατά αλφαβητική σειρά των `utterance_id`.

Στο ερώτημα **4.2.6** δημιουργούμε τα `spk2utt` αρχεία σύμφωνα με τις υποδείξεις της εκφώνησης.

Για το ερώτημα **4.2.7** χρησιμοποιήσαμε τον βασικό βρόχο του αρχείου `local/timit_format_data.sh`, ώστε να δημιουργήσουμε το FST της γραμματικής (`G.fst`). Στην περίπτωση του unigram μοντέλου, το output του `fstisstochastic` είναι ίσο με "`0.00103735 0.00103735`", ενώ για το bigram μοντέλου γίνεται "`0.00141961 -0.076427`". Όπως αναμενόταν ο πρώτος αριθμός του `fstisstochastic` output λαμβάνει σχετικά μικρές, μη μηδενικές όμως τιμές, οπότε το FST δεν είναι τέλεια στοχαστικό. Επίσης, στο bigram μοντέλο ο δεύτερος αριθμός του `fstisstochastic` είναι αρνητικός λόγω των backoff weights, σε αντίθεση με το unigram μοντέλο.

```

for lm_suffix in bg; do
test-data/lang_test_${lm_suffix}
mkdir -p $test
cp -r data/lang/* $test

gunzip -c $tandir/lm_phone_${lm_suffix}.arpa.gz | \
arpa2fst --disambig-symbol=#0 \
--read-symbol-table=$test/words.txt - $test/G.fst
fstisetoochastic $test/G.fst
# The output is like:
# 9.14233e-05 -0.259933
# we do expect the first of these 2 numbers to be close to zero (the second is
# nonzero because the backoff weights make the states sum to >1).
# Because of the <s> fiasco for these particular LMs, the first number is not
# as close to zero as it could be.

# Everything below is only for diagnostic.
# Checking that G has no cycles with empty words (e.g. <s>, </s>):
# this might cause determinization failure of CLG.
# #0 is treated as an empty word.
mkdir -p $tandir/g
awk '{if(NF==1){ print("0 0 %s %s\n", $1,$1); }} END{print "0 0 #0 #0"; print "0:"}' \
< "lexicon" > $tandir/g/select_empty.fst.txt
fstcompile --isymbols=$test/words.txt --osymbols=$test/words.txt $tandir/g/select_empty.fst.txt | \
fstacsort --sort-type=label | fstcompose - $test/G.fst > $tandir/g/empty_words.fst
fstinfo $tandir/g/empty_words.fst | grep cyclic | grep -w 'y' &&
echo "Language model has cycles with empty words" && exit 1
rm -r $tandir/g
done

```

Figure 1: Κύριος βρόχος του αρχείου local/timit\_format\_data.sh

## Ερώτημα 1

Αρχικά, αποσυμπίεσαμε τα .arpa.gz αρχεία μέσω της εντολής "gunzip", ώστε να αποθηκεύσουμε τις πιθανότητες μετάβασης και στη συνέχεια διαβάζοντας τα κείμενα εισόδου υπολογίσαμε το perplexity σύμφωνα με τον τύπο του θεωρητικού μέρους (Θεωρητική Εισαγωγή, 2ο μέρος), οπότε και λάβαμε τα ακόλουθα αποτελέσματα.

```

Text: test, Bigram perplexity: 15.312
Text: validation, Bigram perplexity: 16.447
Text: test, Unigram perplexity: 31.591
Text: validation, Unigram perplexity: 32.035

```

Figure 2: Αποτελέσματα perplexity στο validation και στο test set

Όπως είχαμε αναλύσει και στο θεωρητικό μέρος (Θεωρητική Εισαγωγή, 2ο μέρος), ένα μοντέλο θεωρείται ότι εμφανίζει καλύτερη απόδοση αν παρουσιάζει μικρότερη τιμή perplexity. Επομένως, παρατηρούμε πειραματικά ότι τα bigram μοντέλα εμφανίζουν καλύτερες επιδόσεις από τα unigram μοντέλα. Το συμπέρασμα αυτό ήταν αναμενόμενο καθότι τα unigram μοντέλα εκτιμούν απλώς την πιθανότητα ύπαρξης μίας λέξης στο κείμενο, δίχως να λαμβάνουν υπόψη τους τις συσχετίσεις μεταξύ των λέξεων, σε αντίθεση με τα bigram μοντέλα. Τα μοντέλα αυτά μελετώνται σε μεγαλύτερο βάθος στο θεωρητικό μέρος (Θεωρητική Εισαγωγή, 2ο μέρος).

Αξίζει να σημειώσουμε ότι και στην περίπτωση του bigram και στην περίπτωση του unigram μοντέλου το test set εμφάνισε χαμηλότερο perplexity από το validation set. Αυτό συμβαίνει διότι πιθανότατα το testing set παρουσιάζει λιγότερες διαφορές από το validation set σε σχέση με το train set. Μάλιστα, είχε αναφερθεί στο θεωρητικό μέρος ότι τα N-gram language models παρουσιάζουν καλές επιδόσεις μόνο εάν το test set είναι πανομοιότυπο με το train set.

## 4.3 Εξαγωγή ακουστικών χαρακτηριστικών

Στο ερώτημα 4.3 εξάγουμε τα MFCCs και για τα 3 σετ σύμφωνα με τις οδηγίες τις εκφώνησης.

## Ερώτημα 2

Η Cepstral Mean and Variance Normalization (CMVN) είναι μία μέθοδος κανονικοποίησης που μετασχηματίζει το διάνυσμα χαρακτηριστικών του MFCC ώστε να αποκτήσει μηδενική μέση τιμή και μοναδιαία διακύμανση. Γενικότερα, η κανονικοποίηση των MFCCs συνεισφέρει στη σταθεροποίηση της διαδικασίας εκπαίδευσης και βελτιώνει την ικανότητα του μοντέλου στη γενίκευση. Ειδικότερα, ο μηδενισμός της μέσης τιμής περιορίζει τον θόρυβο, δηλαδή την επίδραση τυχόν σταθερών που έχουν προστεθεί στους cepstral coefficients, ενώ αντίθετα χάρη στη μοναδιαία διακύμανση κάθε χαρακτηριστικό έχει το ίδιο δυναμικό εύρος.

Συνεπώς, χρησιμοποιούμε τη μέθοδο CMVN για λόγους γενίκευσης, ανθεκτικότητας ως προς τον θόρυβο και αριθμητικής ευστάθειας. Οι κανονικοποιημένοι CMVN συντελεστές, ορίζονται ως

$$\hat{c}_t(d) = \frac{c_t(d) - \mu_d}{\sigma_d}$$

όπου

$$\mu_d = \frac{1}{T} \sum_{t=1}^T c_t(d) \text{ και } \sigma_d = \sqrt{\frac{1}{T} \sum_{t=1}^T (c_t(d) - \mu_d)^2}$$

Η κανονικοποίηση CMVN μπορεί να γίνει με διαφορετικές μεθόδους, όπως η Utterance-level CMVN, όπου η μέση τιμή και η διακύμανση υπολογίζονται ανά utterance, ή η Global CMVN, όπου η μέση τιμή και η διακύμανση υπολογίζονται σε όλο το training dataset. Η μέθοδος που περιγράφηκε μαθηματικά είναι η Global CMVN.

### Ερώτημα 3

Προκειμένου να απαντήσουμε στο συγκεκριμένο ερώτημα, γράφουμε ένα bash script η έξοδος του οποίου δίνεται ακολούθως.

```
Frames for first 5 training utterances:
feat-to-len scp:data/train/feats.scp ark,t:-
#1_0 317
#1_1 371
#1_10 388
#1_100 393
#1_101 495
MFCC Feature Dimension:
copy-feats scp:/home/ilili9/kaldi/egs/usc/mfcc/raw_mfcc_train.1.scp ark,t:-
LOG (copy-feats[5.5.1182-1-e02e3]:main():copy-feats.cc:143) Copied 264 feature matrices.
MFCC dimension: 13
```

Figure 3: Έξοδος script για το ερώτημα 3.

Αξίζει να σημειώσουμε ότι οι συντελεστές του MFCC είναι 13 σύμφωνα με τον κώδικα. Αυτό συμβαίνει διότι με την εντολή "make\_mfcc.sh" υπολογίζονται μόνο οι 13 συντελεστές του mel-cepstrum, καθώς οι delta cepstral coefficients και double delta cepstral coefficients μπορούν να εξαχθούν εύκολα από αυτούς, όπως αναλύσαμε στο θεωρητικό μέρος (**Θεωρητική Εισαγωγή, 1ο μέρος**).

#### 4.4 Εκπαίδευση ακουστικών μοντέλων και αποκωδικοποίηση προτάσεων

Για το **4.4.1** χρησιμοποιήθηκε το bash script του kaldi train\_mono.sh που βρίσκεται στον φάκελο steps. Το script αυτό εκπαιδεύει ένα monophone ακουστικό μοντέλο με βάσει τα training data. Αξιοσημείωτη είναι η χρήση του option -boost\_silence και συγκεκριμένα στην εντολή μας -boost\_silence 1.25. Αυτό σημαίνει πως στα πλαίσια σιγής δίνεται βάρος 1.25 μεγαλύτερο σε σχέση με τα υπόλοιπα φωνήματα, ώστε να κάνουμε το μοντέλο μας πιο ικανό στο να καταλαβαίνει τότε υπάρχει σιγή. Εκπαιδεύουμε 2 μοντέλα με αυτή την μέθοδο, ένα για unigram και ένα για bigram γλωσσικά μοντέλα.

Για το **4.4.2** έγινε χρήση του bash script mkgraph.sh του Kaldi, που βρίσκεται στον φάκελο utils. Με αυτό κάνουμε generate το HCLG, για το οποίο θα μιλήσουμε εκτενέστερα στην απάντηση του ερωτήματος 6. Όπως και στο πρώτο ερώτημα, εκτελείται 2 φορές, μια για unigram και μια για bigram.

Για το **4.4.3** κάνουμε χρήση της εντολής decode.sh που βρίσκεται στον φάκελο utils. Την εκτελούμε και 2 φορές για unigram και bigram και για το test και για το dev, άρα συνολικά 4 φορές. Αξιοσημείωτο είναι το γεγονός πως το kaldi καλεί την συνάρτηση "python" και όχι "python3", οπότε μέσα στο script έχουμε βάλει να γίνεται soft link του python3 στο python, με επίδραση το script να χρειάζεται sudo για να εκτελεστεί.

Το **4.4.4** μπορεί να παραληφθεί εάν έχουμε μετονομάσει το "score\_kaldi.sh" σε "score.sh" μέσα στον φάκελο local. Οι 2 υπερπαραμέτροι που υπάρχουν στο scoring είναι οι τιμές του deletion και του substitution σε σχέση με το insertion, υπάρχουν πολλές περιπτώσεις που δεν μπορούμε να θεωρήσουμε όλα τα είδη λαθών ισοπίθانا. Τα αποτελέσματα που πήραμε από το mono παρουσιάζονται μερικώς στους παρακάτω πίνακες:

Table 1: Mono Val Unigram

| num          | ins | del  | sub  | phones | PER     |
|--------------|-----|------|------|--------|---------|
| 7_0.0 (best) | 156 | 1310 | 1123 | 4499   | 57.55 % |
| 8_0.0        | 152 | 1452 | 1025 | 4499   | 58.44 % |
| 9_0.0        | 132 | 1573 | 969  | 4499   | 59.44%  |
| 10_0.0       | 119 | 1685 | 914  | 4499   | 60.41 % |

Table 2: Mono Test Unigram

| num          | ins | del  | sub  | phones | PER     |
|--------------|-----|------|------|--------|---------|
| 7_0.0 (best) | 395 | 4005 | 2674 | 12006  | 58.92 % |
| 8_0.0        | 332 | 4323 | 2532 | 12006  | 59.86%  |
| 9_0.0        | 291 | 4640 | 2361 | 12006  | 60.74%  |
| 10_0.0       | 252 | 5004 | 2179 | 12006  | 61.93%  |

Table 3: Mono Val Bigram

| num          | ins | del  | sub  | phones | PER     |
|--------------|-----|------|------|--------|---------|
| 7_0.0 (best) | 246 | 977  | 1160 | 4499   | 52.97%  |
| 8_0.0        | 217 | 1075 | 1114 | 4499   | 53.48%  |
| 9_0.0        | 190 | 1158 | 1095 | 4499   | 54.30%  |
| 10_0.0       | 179 | 1240 | 1071 | 4499   | 55.35 % |

Table 4: Mono Test Bigram

| num          | ins | del  | sub  | phones | PER     |
|--------------|-----|------|------|--------|---------|
| 7_0.0 (best) | 579 | 3132 | 2743 | 12006  | 53.76%  |
| 8_0.0        | 519 | 3321 | 2687 | 12006  | 54.36%  |
| 9_0.0        | 444 | 3504 | 2615 | 12006  | 54.66 % |
| 10_0.0       | 395 | 3730 | 2525 | 12006  | 55.39 % |

Στο **4.4.5** χρησιμοποιούμε το mono μοντέλο που φτιάξαμε στα παραπάνω ερωτήματα ώστε να κάνουμε align (μέσω του αρχείου align\_si.sh) , έπειτα το εκπαιδεύουμε με το αρχείο train\_delta.sh (επομένως θα έχουμε και πληροφορία της πρώτης παραγωγού των ηχητικών σημάτων). Μετά από αυτό τα βήματα είναι όμοια με το 4.2 έως το 4.4. Παρακάτω είναι ένα διάγραμμα που συγκρίνει τα καλύτερα μοντέλα mono και tri ανάλογα το dataset και το εάν χρησιμοποιήθηκε unigram ή bigram.



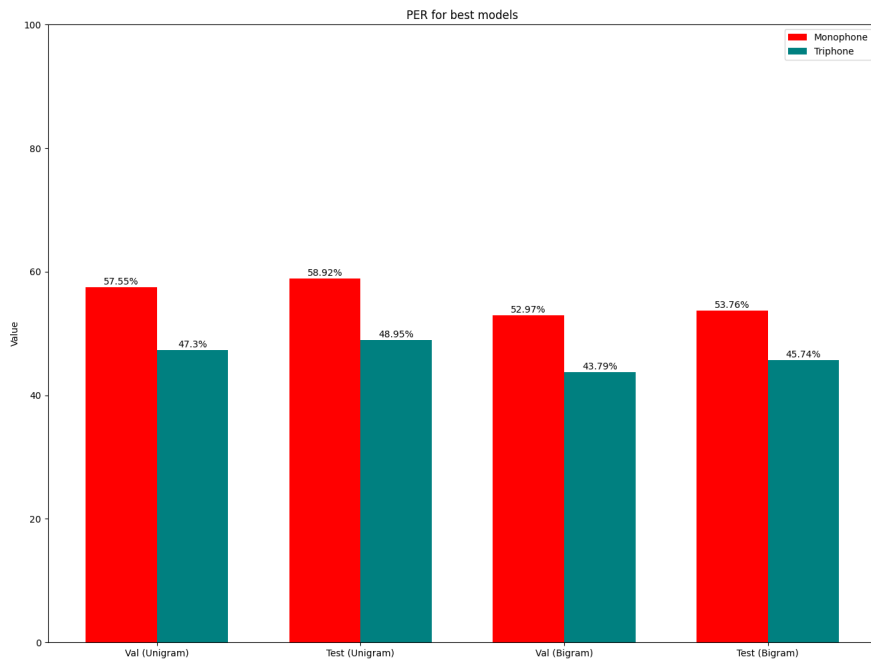


Figure 4: Σύγκριση αποτελεσμάτων Monophone και Triphone

#### Ερώτημα 4

Καθώς εξηγήσαμε αρκετά αναλυτικά τα GMM-HMM στο θεωρητικό μέρος (**Θεωρητική Εισαγωγή, 3ο μέρος**) σε αυτή την απάντηση θα δώσουμε παραπάνω βάση στην εκπαίδευση των GMM-HMM. Η εκπαίδευση γίνεται συνήθως σε 2 στάδια. Στο πρώτο στάδιο, για κάθε κατάσταση του HMM (που αρχικά είναι τυχαίες) εκπαιδεύουμε το GMM χρησιμοποιώντας αλγορίθμους Expectation-Maximization σε ηχητικά χαρακτηριστικά (συνήθως MFCC) που σχετίζονται με το φώνημα. Οι Γκαουσιανές παίρνουν κατάλληλες μέσες τιμές, διακυμάνσεις και βάρη, ώστε να μπορούν να ακολουθήσουν την κατανομή των φωνημάτων της κάθε κατάστασης. Στο δεύτερο στάδιο, προσπαθούμε να βρούμε τις πιθανότητες μεταβάσεων καταστάσεων του HMM (συνήθως με αλγόριθμο Baum-Welch) και μετά ξαναπερνάμε από τις παραμέτρους του GMM, προσπαθώντας να αλλάξουμε τις τιμές ώστε να μεγιστοποιείται η πιθανοφάνεια των ηχητικών δεδομένων που έχουμε. Ότι είπαμε ισχύει και για το μονοφωνικό, ωστόσο το μονοφωνικό, καθώς λαμβάνει υπόψιν μόνο το τρέχον φώνημα για να βγάλει συμπεράσματα είναι πολύ πιο γρήγορο στο να υπολογιστεί και επομένως χρησιμοποιείται για να κάνουμε align, δηλαδή αντί για τυχαίες αρχικές καταστάσεις στο HMM, να τις υπολογίσουμε μέσω του μονοφωνικού μοντέλου, κάνοντας "bootstrapping" στα μεγαλύτερα ακουστικά μοντέλα.

#### Ερώτημα 5

Η a posteriori πιθανότητα στην αναγνώριση φωνής δίνεται από τον παρακάτω τύπο:

$$P(W|X) = \frac{P(X|W)P(W)}{P(X)} = \frac{P_A(X|W)P_L(W)}{P(X)}$$

Όπου:

- $X$  είναι ένα διάνυσμα ακουστικών χαρακτηριστικών (συνήθως MFCC)
- $W$  το φώνημα που θέλουμε να αναγνωρίσουμε
- $P(W|X)$  η πιθανότητα να έχουμε το φώνημα  $W$  από το διάνυσμα χαρακτηριστικών  $X$
- $P(Q|W)$  η πιθανότητα να παρατηρήσουμε το διάνυσμα  $X$  όταν έχουμε το φώνημα  $W$  (αποτελεί το ακουστικό μοντέλο μας)
- $P(W)$  η πιθανότητα εμφάνισης του φωνήματος a priori (αποτελεί το γλωσσικό μοντέλο μας)
- $P(X)$  η πιθανότητα να παρατηρήσουμε το  $X$ , δρά απλά σαν κανονικοποίηση, μπορούμε να το αγνοήσουμε

Για να βρούμε την πιο πιθανή λέξη σύμφωνα με το διάγραμμα χαρακτηριστικών μας, πολύ απλά παίρνουμε:

$$\hat{W} = \underset{W}{\operatorname{argmax}} P(W|\mathbf{X}) = \underset{W}{\operatorname{argmax}} P(\mathbf{X}|W)P(W)$$

Οπότε αφού  $P(W)$  γνωστό (a priori πιθανότητα του φωνήματος), αρκεί να υπολογίσουμε για όλα τα πιθανά  $W$  την πιθανότητα  $P(W|\mathbf{X})$  και μετά να δούμε για ποιο  $W$  μεγιστοποιείται το γινόμενο τους. Αυτό θα είναι το φώνημα που θα επιλέξουμε.

## Ερώτημα 6

Στο kaldi το HCLG είναι ένα FST (Finite State Transducer) με βάρη, συγκεκριμένα είναι ένα decoding graph που προκύπτει από την σύνθεση 4 FST,  $HCLG = H \circ C \circ L \circ G$

Όπου

- H: HMM, Αναπαριστά την ηχητική ακολουθία σε ακολουθία καταστάσεων, με πιθανότητες μετάβασης από την μια στην άλλη
- C: context-dependency, αντιστοιχίζει τα φωνήματα που έχουν εξαρτήσεις από συμφραζόμενα (όπως π.χ. στα triphones) σε ανεξάρτητα των συμφραζωμένων (monophone)
- L: Lexicon, είναι το λεξικό προφοράς και αντιστοιχίζει ακολουθίες φωνημάτων (phones) σε λέξεις. Μπορεί να περιλαμβάνει πολλαπλές προφορές για την ίδια λέξη, με αντίστοιχα βάρη (πιθανότητες)
- G: Το γλωσσικό μοντέλο (εδώ είτε unigram είτε bigram), αναθέτει πιθανότητες σε κάθε ακολουθία φωνημάτων.

Άρα με απλά λόγια το G εμπεριέχει πληροφορία για το ποιες λέξεις είναι πιθανές, το L για το πως γράφονται σε φωνήματα, το C για το πως αλλάζουν τα φωνήματα σύμφωνα με τα συμφραζόμενα και το H για το πως φαίνονται σε ηχητικά σήματα, ανά πλαίσιο.

## 4.5 Μοντέλο DNN-HMM με PyTorch

Για όλα τα βήματα έχει προστεθεί ο κώδικας οποιουδήποτε προγράμματος από το φάκελο [lab2](#) του github χρειάστηκε να αλλάχθει, καθώς και ένα Bash Script . Σημειώνουμε πως το alignment έγινε μόνο με το triphone με bigram, λόγω του χρόνου εκτέλεσης που απαιτεί η εκπαίδευση (δεν είχαμε επαρκή χρόνο να αφιερώσουμε για να επαναλάβουμε και για το unigram). Παρακάτω παρουσιάζουμε τα αποτελέσματα μας στο test dataset για μερικές δοκιμές υπερπαραμέτρων του DNN, αλλάξαμε μόνο το πλήθος των layers και τους νευρώνες ανά layer, δεν παρουσιάζουμε όλες μας τις δοκιμές:

Table 5: Αποτελέσματα DNN-HMM στο Test

| neurons per layer | layers | num   | ins | del  | sub  | phones | PER     |
|-------------------|--------|-------|-----|------|------|--------|---------|
| 256               | 2      | 3_0.0 | 776 | 1587 | 2195 | 10093  | 45.16%  |
| 256               | 3      | 5_0.0 | 591 | 1933 | 2112 | 10093  | 45.93 % |
| 512               | 2      | 3_0.0 | 864 | 1515 | 2091 | 10093  | 44.29%  |
| 512               | 3      | 4_0.0 | 748 | 1688 | 2040 | 10093  | 44.35%  |
| 1024              | 2      | 5_0.0 | 600 | 1769 | 1968 | 10093  | 42.97%  |
| 1024 (best)       | 3      | 4_0.0 | 786 | 1592 | 1957 | 10093  | 42.95 % |
| 2048              | 2      | 2_0.5 | 939 | 1463 | 2060 | 10093  | 44.21 % |

Παρατηρούμε πως η διαφορά μεταξύ του PER στο μοντέλο με 1024 νευρώνες ανά επίπεδο με 2 επίπεδα και 3 επίπεδα είναι μόλις 0.02%.

## Ερώτημα 7

Καθώς εξηγήσαμε αρκετά αναλυτικά τα DNN-HMM στο θεωρητικό μέρος ([Θεωρητική Εισαγωγή, 3ο μέρος](#)) σε αυτή την απάντηση θα δώσουμε παραπάνω βάση στο εάν είναι εφικτό να κατασκευάσουμε ένα DNN-HMM χωρίς bootstrapping. Η απάντηση είναι πως ναι, υπάρχουν και άλλοι τρόποι να γίνει το aligning των πλαισίων, όπως το Connectionist Temporal Classification (CTC), όπου το aligning μαθαίνεται την ώρα της εκπαίδευσης. Όμως, εάν μείνουμε στις μέχρι τώρα γνώσεις μας και δεν πάμε να αναζητήσουμε στην βιβλιογραφία, η απάντηση είναι μάλλον όχι. Διότι αρχικά ο διαχωρισμός θα

γίνει εντελώς τυχαία και επομένως εάν προσπαθούσαμε να βγάλουμε συμπεράσματα για το aligning, τότε θα ήταν πολύ πιθανό να καταλήγαμε σε ένα πολύ κακό τοπικό ελάχιστο, από το οποίο δεν θα ξεφεύγαμε. Ενδεχομένως να μπορούσαν να κατασκευαστούν ευρετικά κριτήρια που θα μας έβρισκαν "καλές" αρχικοποιήσεις, αλλά σε καμία περίπτωση δεν θα μπορούσαν να το εγγυηθούν στην γενική περίπτωση. Όσον αφορά το πότε μας συμφέρει η προσθήκη DNN αντί για GNN, η απάντηση είναι σχετικά απλή, όποτε δεν έχουμε περιορισμό πόρων, τότε κατά πάσα πιθανότητα το DNN-HMM θα έχει παραπάνω ακρίβεια από ένα αντίστοιχο μοντέλο GMM-HMM, ωστόσο είναι πιο περίπλοκα στην κατασκευή τους και φυσικά όπως αναφέραμε λίγο παραπάνω, πάλι θα χρειαζόμαστε ένα GNN-HMM, απλώς ως ενδιάμεσο βήμα και όχι ως τελικό εκτιμητή.

## Ερώτημα 8

Η τεχνική του Batch Normalization έχει ως στόχο να λύσει το πρόβλημα του internal covariate shift, δηλαδή το γεγονός πως σε ένα layer αλλάζουν πολύ συχνά οι διακυμάνσεις των δεδομένων. Αυτό έχει ως αποτέλεσμα να χρειαζόμαστε μικρότερο βήμα εκπαίδευσης (άρα παραπάνω χρόνο εκπαίδευσης και παραπάνω κόστος λειτουργίας) καθώς μπορεί να έχουμε αστάθειες λόγω διαφορετικών διακυμάνσεων. Στο Batch Normalization κανονικοποιούμε τις εισόδους πριν τις εισάγουμε στο layer και επομένως δεν έχουμε μεγάλες αλλαγές διακύμανσης. Επιπλέον μπορεί να εξελιχθεί περαιτέρω και να εισαχθούν παράμετροι για κλιμάκωση και μετατόπιση που είναι learnable και καθορίζονται την ώρα της εκπαίδευσης, επιτρέποντας έτσι το ίδιο το μοντέλο να βρει την κατάλληλη διακύμανση των δεδομένων εισόδου σε κάθε layer. Στα θετικά έχουμε την μεγαλύτερη σταθερότητα, τη γρηγορότερη σύγκλιση, την μείωση εξαρτήσεως από την αρχικοποίηση των βαρών και ένα indirect regularization. Στα αρνητικά έχουμε το γεγονός πως αυξάνεται το πλήθος των πράξεων που χρειάζεται να υπολογίσουμε, κάτι που στις περισσότερες περιπτώσεις μπορούμε να αναπληρώσουμε ή και συχνά να γλυτώσουμε χρόνο, από την αύξηση του learning rate. Επομένως το εάν χρειάζεται Batch Normalization έχει να κάνει με το πόσο δομημένα είναι ήδη τα δεδομένα. Στα δεδομένα πραγματικού κόσμου, τα οποία συχνά περιέχουν έντονο θόρυβο, το Batch Normalization προσφέρει παραπάνω οφέλη από ότι κόστη στη σύννητη περίπτωση.

Στη γενική περίπτωση η είσοδος στο layer θα έχει κανονικοποιηθεί σε:

$$y_i = \gamma \hat{x}_i + \beta$$

Όπου:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \text{ με } \epsilon \text{ μια μικρή σταθερά για να αποφύγουμε την διαίρεση με το μηδέν}$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

## 3. ΚΩΔΙΚΑΣ

Ο κώδικας είναι στον φάκελο NLP\_kaldi\_lab\_code. Περιέχει υποφακέλους για τα μέρη 3, 4.1, 4.2, 4.3, 4.4, 4.5 και μέσα στον κάθε υποφάκελο εμπεριέχονται τα Scripts που τρέξαμε για το κάθε υποερώτημα που αντιστοιχεί στο όνομα του υποφακέλου.